```
  "created_at" datetime,
  "updated_at" datetime
) auto_increment = 10000;
```

Be careful when using the `:options` parameter with MySQL. The Rails MySQL database adapter sets a default option of `ENGINE=InnoDB`. This overrides any local defaults you have and forces migrations to use the InnoDB storage engine for new tables. Yet, if you override `:options`, you'll lose this setting; new tables will be created using whatever database engine is configured as the default for your site. You may want to add an explicit `ENGINE=InnoDB` to the options string to force the standard behavior in this case. You probably want to keep using InnoDB if you're using MySQL because this engine gives you transaction support. You might need this support in your application, and you'll definitely need it in your tests if you're using the default of transactional test fixtures.

Renaming Tables

If refactoring leads us to rename variables and columns, then it's probably not a surprise that we sometimes find ourselves renaming tables too. Migrations support the `rename_table` method:

```
class RenameOrderHistories < ActiveRecord::Migration
  def change
    rename_table :order_histories, :order_notes
  end
end
```

Rolling back this migration undoes the change by renaming the table back.

Problems with rename_table

When we rename tables in migrations, a subtle problem arises.

For example, let's assume that in migration 4 we create the `order_histories` table and populate it with some data:

```
def up
```